

Component Views – Introduction

When designing software using Component-Based Design (CBD), we don't just look at the **code** of a component — we also examine it from **different views** to understand how it fits into the whole system.

Why have multiple views?

- Each view shows a **different aspect** of a component.
 - This helps designers, developers, and testers understand how to build and use the component properly.
- Viewing components from multiple angles helps ensure that the **design is complete, well-structured, and reusable**.

1. Object-Oriented View

- This view looks at the component in terms of classes and objects, following object-oriented principles like encapsulation, inheritance, and polymorphism.
- Focus Area:
 - **Public interfaces** – methods or operations that other components can use.
 - **Private data** – internal data hidden from others.
 - **Class diagrams** – show relationships between internal classes.

Example: Shopping Cart Component

- Class: `CartItem`, `CartManager`
- Public methods: `addItem()`, `removeItem()`, `checkout()`
- These classes are inside the component, but only the **public methods** are exposed to users.

When to use:

- When designing a component with multiple classes and behaviors.
 - When **class structure** and **data relationships** matter.
- The object-oriented view shows how the **internal classes work together**, and what services are **publicly offered** to other components.

2. Conventional View

- The conventional view looks at the component more like a **function block** with clear **input, process, and output**. This is a more traditional, structured view.
- Focus Area:
 - What **input** the component receives.
 - What **output** it produces.
 - What is the **main logic or transformation** it performs?
- This view ignores:
 - Object-oriented design rules.
 - Class structures or methods.

Example: Tax Calculator Component

- Input: income amount
- Process: applies tax rules
- Output: total tax to pay

When to use:

- When the component is **simple**, like a calculation or data converter.
- When class structure isn't important.

➤ The conventional view is useful for **understanding the logic** inside a component, especially for small and straightforward tasks.

3. Process-Related View

- This view focuses on how a component behaves when the **system is running** (runtime behavior).
- Focus Area:
 - How the component **communicates** with other components.
 - What **messages** are sent and received.
 - How the component handles **execution flow** and **timing**.

Example: Notification Component

- Triggered when an event happens (e.g., order placed).
- Sends out emails or mobile alerts.
- Receives data from the order system, processes it, sends output.

When to use:

- For **real-time, event-driven, or interactive** systems.
- When analyzing how components **work together during execution**.

➤ The process view helps you understand how a component **interacts and performs** in a **live running system**.

Component-Based Design – Functionality Problems

- Sometimes functionality problems occur that affect the **quality and correctness** of the final system.
- These problems usually happen when combining multiple components that were not properly designed to work together.

1. Overlapping Functionality

- Occurs where two or more components provide the same functionality.
(This means that **multiple components are doing the same task**, even though only one is needed. It leads to **duplication, waste**, and sometimes **confusion** about which component to use.)
- The designer must decide which component **should do the task** and make sure **only that component does it**.
- From a design view, it is **not good** to let different parts provide the same function, as this can cause problems in system maintenance and upgrades.

Real-life example:

In a system, both the **UserModule** and **AdminModule** have a method to **reset password**. They do the same thing, but are implemented separately — this is overlapping.

Problem:

- Wastes resources
- Makes the system harder to maintain
- Can cause inconsistencies (what if one works differently?)

Fix → Identify overlap during design and **merge into one shared component** when possible.

2. Missing Functionality

- The desired functionality is not provided by any component.
(Some important features or services are **completely missing** in the final design. No component is responsible for doing that job.)

Real-life example:

You're building an online store, and everything is done — but no component handles **sending order confirmation emails**. That's a missing function.

Problem:

- User needs are not fully met
- Can lead to critical system failure
- System may look complete, but important things don't work

Fix → Double-check user requirements and ensure **each required function has a responsible component**.

3. Redundant Functionality

- The same functionality is implemented in different ways by different components.

(More than one component **performs the same task**, but using **different methods or logic**. This is not just duplication — it's inconsistent behavior.)

- Components **may offer extra functions** beyond what they were chosen for.
- The designer can either **use** these extra functions (which may cause overlap) or find a way to **block** them.
- Blocking unwanted functions is usually better, but it can be **hard to do**.

Real-life example:

Component A and Component B both generate PDF reports — but A uses one library, and B uses another. The results look different and confuse users.

Problem:

- Reduces consistency
- Hard to maintain and test
- Confuses users and developers

Fix → Use **shared utility components** to make sure the same function is done in **one consistent way**.

Data-Structure-Centered Design

- This design method focuses mainly on the **data** — not the functions, and not the components. The main idea is:
 “If you get the data structure right, the system will be easier to build around it.”
- Instead of thinking about what the system should do (**functions**) or what it's made of (**components or objects**), this method starts by asking:
 “What kind of data do we need to store, and how is it organized?”
- Once the data structure is clear and well-designed, you build functions around it to manipulate or use that data.

♦ **Emphasis on Data Structure First**

- This method starts by designing the **core data structure** — the main part of the system that holds all the important data.
- The rest of the design (including functions and system behavior) is built around this structure.

Example: If you're building a **Library System**, your data structure might be:

- Books (Title, Author, ISBN, Status)
- Members (Name, ID, BorrowedBooks)

You focus on designing these tables/objects/structures first — then later you write functions like `borrowBook()`, `returnBook()`, etc.

♦ **All Major Functions Access the Data Structure**

- Once the data structure is created, many system functions will use it.

Real-life Style Example: Let's say the central data is a **student record**. Then different functions like:

- `updateStudentInfo()`
 - `printTranscript()`
 - `calculateGPA()`
- ...all work on the same student data structure.

This means the structure becomes the **heart** of the whole system.

♦ Interfaces Built to Fit the Data

- In this method, the **user interface** or **API interface** is created in a way that works naturally with the structure of the data. So the system is easier to build and maintain.
- In this design method, we care **mostly about the structure of data**, and design everything else to fit that structure.

When to Use Data-Structure-Centered Design

Use this approach when:

- The system deals with **complex data**
- The data will be **accessed by many functions**
- The data structure is **unlikely to change frequently**

Best for:

- Library systems
- Student or employee management systems
- Inventory or banking systems

Introduction to AOSD

- AOSD is a modern approach to software development that helps manage **cross-cutting concerns** (features that affect many parts of a system, like logging or security).
- Unlike traditional methods (e.g., OOP), AOSD isolates these concerns into separate modules called **aspects** instead of mixing them with main business logic.
- Works alongside other methods (like OOP) to improve code organization.

Real-Life Example: Imagine building a house:

- **Core Logic** = Building walls, doors (main structure).
- **Cross-Cutting Concerns** = Electrical wiring, plumbing (affect multiple rooms).
AOSD keeps wiring/plumbing separate so changes don't require tearing down walls.

What is a cross-cutting concern?

- It's a functionality that spreads across multiple modules of a program but doesn't belong to just one.
- Example: Logging, Security, Error Handling – these tasks are needed across many parts of an app but don't belong to just one module.
- AOSD allows you to modularize these scattered concerns into separate units (called aspects) so that the main code stays clean and focused

What is an Aspect? (A module that handles a cross-cutting concern.)

An aspect is a modular unit that encapsulates cross-cutting concerns. It can:

- Be static (fixed) or dynamic (change at runtime).
- Contain fields, methods, and rules for where it should apply (pointcuts).
- Be privileged (access private parts of other code if needed).

Exp: A `LoggingAspect` can automatically log method calls without modifying each method.

Role of Aspects

- Aspects **define** cross-cutting logic (e.g., security checks) and **where** to apply it (e.g., before/after certain methods).
- They **auto-integrate** into the system, avoiding manual code changes.
- Improve **maintainability** (e.g., updating logging rules in one place).
- Groups functions that affect many parts of a program.
- Shows where to add these functions in the program.
- Adds these functions to the system automatically.
- Helps keep different concerns separate and clear.
- Makes the software easier to design and maintain.

Exp: Like a security camera system (Aspect) added to a store (Core System):

- Cameras monitor all entrances (cross-cutting).
- No need to modify doors or shelves to add cameras.

Advice

- Action taken by an aspect at specific points (join points).
 1. Before: Runs before a method (e.g., input validation).
 2. After: Runs after a method (e.g., cleanup).
 3. Around: Wraps a method (e.g., performance logging).
 4. After Throwing: Runs if a method fails (e.g., error alerts).

Concerns in Software

- A concern is any feature/requirement affecting the system
- A concern is something important in a system.
- It helps break software into smaller, easier parts.
- A concern is information that affects the code.
- It can be big, like database work, or small, like a simple calculation.

Example: In a banking app:

- Core Concern: Transfer money.
- Cross-Cutting Concern: Fraud detection (applies to transfers, withdrawals, etc.).
- **Quality concerns** are different parts of how software works, like performance, security, or special features.
- These are not just coding issues, but **show what the system needs** and **what users or stakeholders care about**.
- For example, the business logic is one concern, and the user interface is another.

Common concerns include:

- Business Logic (e.g., calculating interest).
- Technical (e.g., database access, logging).
- Operational (e.g., security, multi-threading).

Why It Matters: **Mixing these in code creates spaghetti logic. AOSD separates them cleanly.**

Core vs. Secondary Concerns

Core Concerns

- Central to the system (e.g., online shopping cart).
- These are the main functions that serve the system's main purpose.
- They can be:
 - Business-related tasks
 - Domain-specific tasks
 - Functional requirements

Secondary Concerns

- Supportive (e.g., user authentication, analytics).
- These are about non-functional needs like quality and service performance.

Exp: Core: Adding items to cart.

Secondary: Sending a confirmation email (applies to cart + orders).

Stakeholder Concerns (Different stakeholders care about different concerns)

- **Functional concerns** are about the specific features a system must have. (**Core features**)
- **Quality of Service (QoS) concerns** deal with how well the system performs, like speed or reliability. (**performance, security, reliability**)
- **Policy concerns** involve **rules** and **guidelines** for how the system should be used.
- **System concerns** focus on the overall system, like how easy it is to update or configure.
- **Organisational concerns** relate to business goals, like staying within budget, reusing software, or protecting the company's reputation. (**budget, timeline**)

Why It Matters: AOSD helps address these separately.

- Developers focus on code (functional).
- The Security Team handles encryption (QoS).
- Management ensures deadlines (organizational).

Separation of Concerns (SoC)

- When concerns are kept separate, each part can be reused, built, and updated on its own.
- Separation of concerns means **keeping the code for each task in its own place**.
- Each part should be easy to understand without needing to look at other parts
- Principle: Means each part of the system should handle one specific task. This helps make each part **more independent** and **focused on its own task**.
- Goal: → Make code easier to maintain, reuse, and update.
→ To split software into clear parts that don't overlap much.

Real-Life Example:

- A restaurant divides work among chefs (cooking), waiters (serving), and cleaners (hygiene). No one does everything!
- (changing the user interface shouldn't affect the business logic, and the other way around.)
- Program features like subroutines, procedures, and objects help separate concerns.
- This separation makes it easier to **trace how requirements are turned into code**.
- The Model-View-Controller (MVC) design is a good example of this, helping make software easier to maintain.
 - Model: Data logic (e.g., database).

- View: UI (e.g., web page).
- Controller: Middleman (handles user input).
- Designers work on View, developers on Model, and no one steps on each other's toes.

Advantages /Usage

- **Make software simpler and easier to understand**
 - Each part of the system focuses on one specific task (e.g., UI, data handling, security).
 - Example: Just like a car's engine (power) and steering (control) are separate, SoC keeps code organized.
- **Improve tracking of changes and requirements across the development process**
 - Clear links between requirements and code modules.
 - Example: If a "Login Security" rule changes, you only check the authentication module—no need to dig through unrelated code.
- **Reduces Impact of Changes (Easier Updates & Customization)**
 - Modify one part without breaking others.
 - Example: Changing a website's color scheme (UI) won't affect how data is stored (database).
- **Support reuse of parts**
 - Modules can be plugged into other projects.
 - Example: A payment processing module reused in an e-commerce app and a subscription service.
- **Make it easier to combine components**
 - Independent parts work together like Lego blocks.
 - Example: Adding a "Google Maps" service to a food delivery app without rewriting the entire system.

Cross-Cutting Concerns

- Not all concerns fit neatly into one component – some are cross-cutting.
- Cross-cutting concerns **affect many parts** of the program.
- This makes **changes harder**, as the related code is spread across the system.
- They also **increase** the system's **complexity**.

Definition: Features that "cut across" multiple modules (e.g., logging, security).

Problem: Mixing them with core logic creates messy code.

Example: Adding "audit logs" to every transaction in a bank app—without AOSD, you'd modify every transaction method!

Scattering Problem

- When the same or similar code is spread across many parts of a program, it's called **code scattering**.
- This means related code is found in many places, not in one spot.
- Unlike shared components, this can lead to **mistakes and inconsistencies**.
- If the code needs to change, you have to **find and update it everywhere**.
- *For example, code for things like authentication, logging, or contract checks often appears in many methods, classes, and packages.*
- *If logging format changes, you must update 100+ methods—error-prone!*

Tangling

- Sometimes, a module handles several requirements at once.
- When multiple concerns are mixed in the same code, it becomes harder to understand — this is called **code tangling**.
- Changing one part may accidentally affect others.
- One module handles multiple concerns (e.g., a method does "payment + logging + validation")
- *For example, a function like `someOperations()` might do more than its main task by including extra concern-related code.*
- *A `saveUser()` method that also sends emails, checks permissions, and logs errors—too many responsibilities!*
- **Result:** Unreadable, rigid code.

Benefits of Aspect-Oriented Software Development (AOSD)

1. Better Modularity

- AOSD allows separating cross-cutting concerns (like logging, security) into their own modules called **aspects**, instead of mixing them with core business logic.
- This keeps the system more organized and easier to manage.

2. Improved Maintainability

- If I need to update a cross-cutting concern (e.g., change how logging works), just only modify the aspect — not every module.
- This makes future changes faster and reduces the risk of errors.

3. Higher Code Reusability

- Aspects can be written once and reused in multiple places throughout the system.
- For example, a single error handling aspect can be applied to many functions without rewriting it.

4. Clearer Focus on Core Logic

- By moving unrelated logic (like authentication or monitoring) into aspects, the main modules can focus only on their **primary responsibility**.
- This results in cleaner, simpler code that's easier to read and understand.

Exp: If you create a logging aspect and apply it to all payment-related functions, you only need to update that aspect to change how logging works — no need to touch the payment code itself.

Conclusion:

AOSD helps build cleaner, reusable, and more maintainable software, especially in large systems with many shared features.

Service-Oriented Architecture (SOA)

- Building systems as reusable services (e.g., web APIs).
- This is a way to **build distributed software** using web services on different computers.
- Services: Independent, communicate via messages (e.g., REST).
 - The software is divided into a group of services.
 - A service is a clear, independent function that doesn't rely on other services' state.
 - A service is the connection point and runs on a system that supports it.
 - Services are loosely connected and keep their internal workings hidden.
 - An application can combine many services to perform tasks.

Real-Life Example: Food delivery app uses

→ Payment Service (Stripe) , Map Service (Google Maps).

→ Each service can be updated without affecting others.

- These services communicate with each other by either:
 - Simply passing data, or
 - Working together to complete a task.

To do this, there must be a way to **connect the services** so they can interact properly.

1. AOSD manages cross-cutting concerns (logging, security) via aspects. 通过方面管理横切关注点
2. Separation of Concerns keeps code modular (like MVC). 关注点分离使代码保持模块化
3. SOA builds systems from reusable, independent services. 使用可重用的独立服务构建系统

Memorization Tip:

- AOSD = "Plugins" for cross-cutting features.
- SOA = "Lego blocks" for scalable systems.

Final Example:

Imagine an e-commerce app:

- AOSD: Logs all orders (without modifying order logic). 记录所有订单(不修改订单逻辑)
- SOA: Uses external "Fraud Detection" and "Shipping" services. 使用外部的“欺诈检测”和“配送”服务。